

Scheduling algorithms (OS)

Name: GUTTIKONDA GAYATHRI

Roll No: 2520030465

Sec: 2

1.FCFS Scheduling algorithm

Code:

```
GNU nano 8.7.1 fcfs.c
#include <stdio.h>
int main(){
    int n, i;
    int bt[20], wt[20], tat[20];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time for each process:\n");
    for(i = 0; i < n; i++){
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for(i = 1; i < n; i++){
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++){
        tat[i] = wt[i] + bt[i];
    }
    printf("\nprocess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++){
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    return 0;
}
```

Output:

```
gayathri@Gayathri: ~$ nano fcfs.c
gayathri@Gayathri: ~$ gcc fcfs.c -o fcfs
gayathri@Gayathri: ~$ ./fcfs
Enter number of processes: 2
Enter burst time for each process:
P1: 5
P2: 3

process BT      WT      TAT
P1      5         0         5
P2      3         5         8
```

2. SJF_NONpreemptive

Code:

```
GNU nano 8.7.1 sjf_nonpreemptive.c *
#include <stdio.h>
int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }
    return 0;
}
```

Output:

```
gayathri@Gayathri:~$ nano sjf_nonpreemptive.c
gayathri@Gayathri:~$ gcc sjf_nonpreemptive.c -o sjf_nonpreemptive
gayathri@Gayathri:~$ ./sjf_nonpreemptive
Enter number of processes: 4
Enter burst time:
P1: 6
P2: 8
P3: 7
P4: 3

Process BT      WT      TAT
P1      3        0        3
P2      6        3        9
P3      7        9       16
P4      8       16       24
```

3.Priority

Code:

```
GNU nano 8.7.1
#include <stdio.h>
int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time and priority:\n");
    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
            i + 1, bt[i], priority[i], wt[i], tat[i]);
    }
    return 0;
}
```

Output:

```
gayathri@Gayathri:~$ nano priority.c
gayathri@Gayathri:~$ gcc priority.c -o priority
gayathri@Gayathri:~$ ./priority
Enter number of processes: 3
Enter burst time and priority:
P1 Burst Time: 5
P1 Priority: 13
P2 Burst Time: 13
P2 Priority: 9
P3 Burst Time: 9
P3 Priority: 6

Process BT      Priority      WT      TAT
P1      9          6           0        9
P2     13          9           9       22
P3      5         13          22       27
```

4. ROUND ROBIN

Code:

```
GNU nano 8.7.1
#include <stdio.h>
int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }
    printf("Enter time quantum: ");
    scanf("%d", &quantum);
    while (1) {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
        if (done == 1)
            break;
        for (i = 0; i < n; i++) {
            tat[i] = wt[i] + bt[i];
        }
        printf("\nProcess\tBT\tWT\tTAT\n");
        for (i = 0; i < n; i++) {
            printf("P%d\t%d\t%d\t%d\n",
                i + 1, bt[i], wt[i], tat[i]);
        }
        return 0;
    }
}
```

Output:

```
gayathri@Gayathri:~$ nano round_robin.c
gayathri@Gayathri:~$ gcc round_robin.c -o round_robin
gayathri@Gayathri:~$ ./round_robin
Enter number of processes: 5
Enter burst time:
P1: 10
P2: 5
P3: 8
P4: 6
P5: 12
Enter time quantum: 3

Process BT      WT      TAT
P1      10      28      38
P2       5      15      20
P3       8      26      34
P4       6      20      26
P5      12      29      41
```